



Taller de Docker

Entregables:

1. Motores DB docker
2. Bases de datos del proyecto en docker
3. Backend y Frontend IIS apuntando a la base de datos Docker
4. Backend y Frontend en Docker apuntando a la base de datos Docker

1. Motores de Bases de Datos en Docker

Vista general de contenedores en Docker Desktop: En la captura se ve el panel Containers de Docker Desktop con los motores de base de datos ejecutándose en contenedores separados. Esta vista confirma que los contenedores están arriba y listos para aceptar conexiones externas desde herramientas cliente o desde aplicaciones.

The screenshot shows the Docker Desktop interface. The left sidebar contains navigation options: Ask Gordon, Containers, Images, Volumes, Builds, Models, MCP Toolkit, Docker Hub, Docker Scout, and Extensions. The main panel is titled 'Containers' and shows a summary of container usage: CPU usage at 1.27% / 1200% and memory usage at 505.24MB / 7.4GB. Below this is a table of running containers.

	Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
☐	determined_noyce	dd287662e82a	postgres:latest		0%	8 minutes ago	[Icons]
☒	motoresdocker	-	-	-	1.27%	1 minute ago	[Icons]
☐	mysql_clean	0a9696c9d5bd	mysql:latest	3308:3306	1.27%	24 minutes ago	[Icons]
☐	sqlserver_clean	490fa10954fb	mssql/server:2022	http://localhost:5434	0%	1 minute ago	[Icons]
☐	postgres_clean	9c711ed7fa23	postgres:latest	5434:5432	0%	24 minutes ago	[Icons]

Showing 5 items

Walkthroughs

- Multi-container applications (8 mins)
- Containerize your application (3 mins)

View more in the Learning center

Engine running | RAM 5.69 GB | CPU 2.93% | Disk: 5.76 GB used (limit 1006.85 GB) | Terminal | New version available

Conexión a PostgreSQL

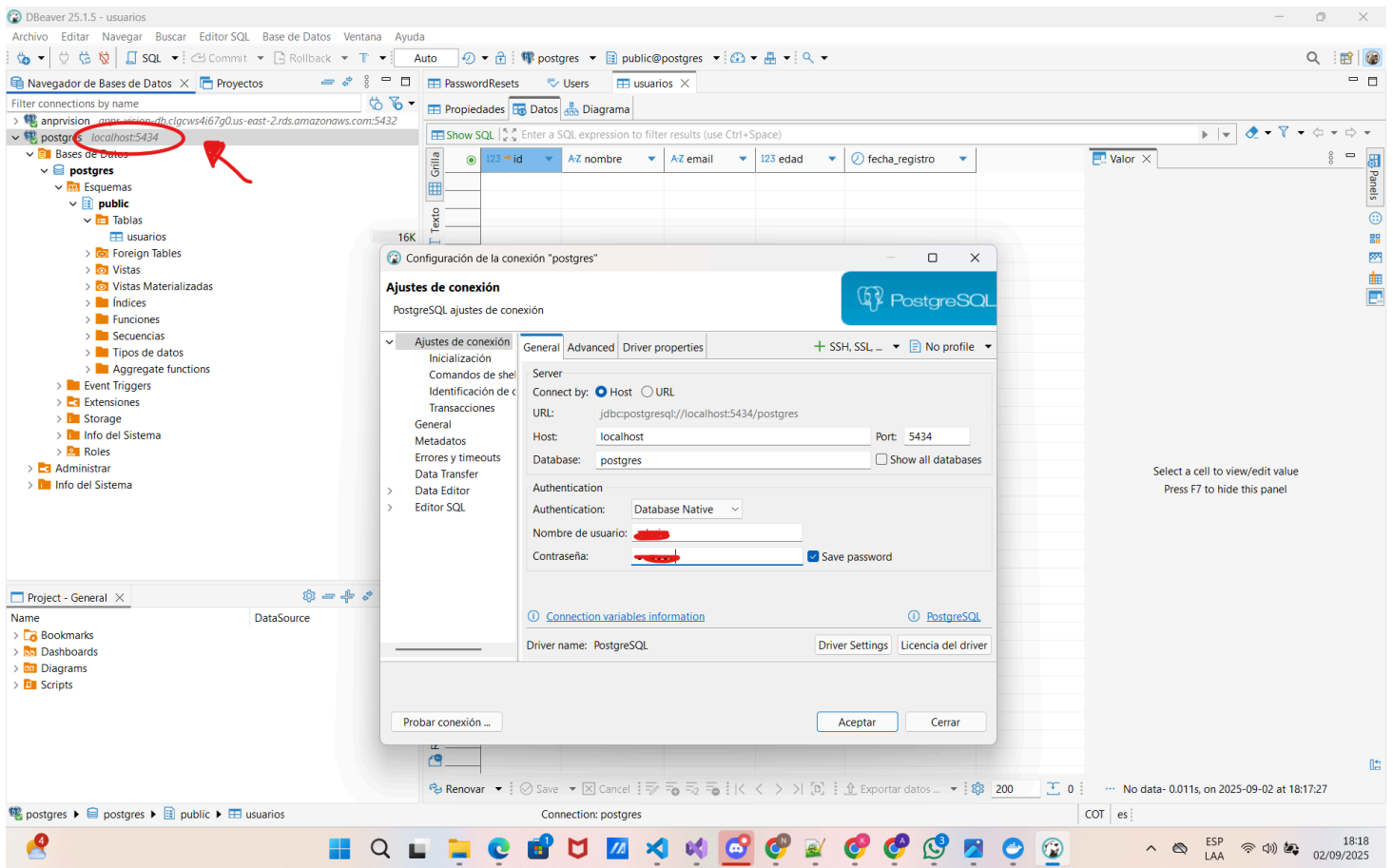
La parte superior muestra los logs del contenedor de PostgreSQL indicando que el servidor se levantó correctamente y quedó a la espera de conexiones. Abajo se ve la conexión configurada en el cliente (DBeaver/pgAdmin) usando Host localhost, Puerto 5434, y las credenciales definidas en el contenedor. Esta combinación verifica que el mapeo de puertos es el correcto y que el motor en Docker responde como si fuera una instalación local, permitiendo explorar esquemas, tablas y ejecutar consultas sin salir del entorno de contenedores.

The screenshot shows the Docker Desktop interface. On the left is a sidebar with navigation options: Ask Gordon, Containers, Images, Volumes, Builds, Models, MCP Toolkit, Docker Hub, Docker Scout, and Extensions. The main area displays the 'postgres_clean' container, which is running. The 'Logs' tab is selected, showing a detailed log of the PostgreSQL server's startup and operation. The logs indicate that the server started successfully on 2025-09-02 at 22:57:46.973 UTC, listening on IPv4 address '0.0.0.0', port 5432, and on IPv6 address ':::', port 5432. It also shows the server is ready to accept connections. The status bar at the bottom indicates the engine is running, with RAM usage at 5.69 GB and CPU usage at 0.42%.

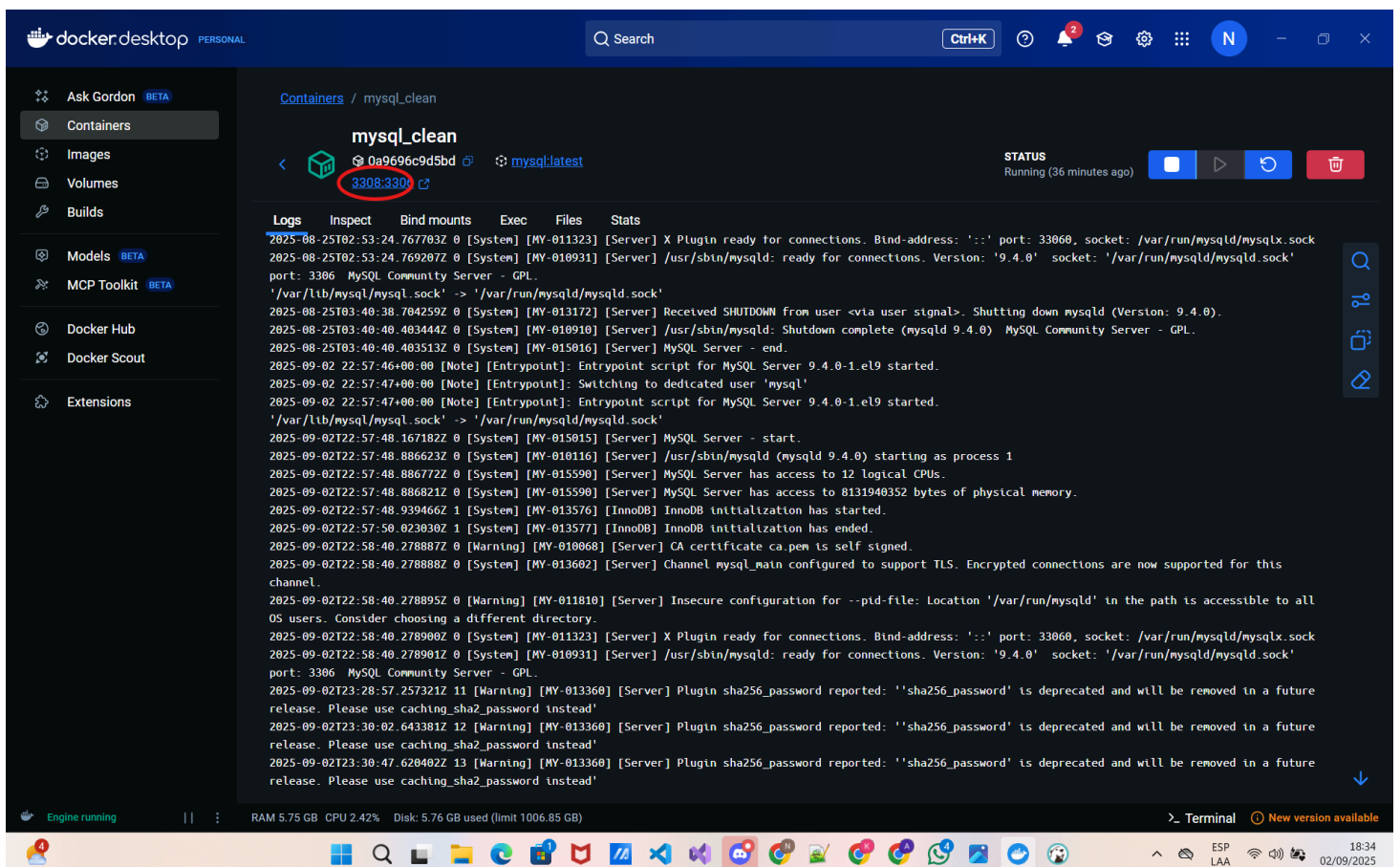
```
2025-08-25 03:09:21.861 UTC [63] LOG:  checkpoint complete: wrote 931 buffers (5.7%); 1 WAL file(s) added, 0 removed, 0 recycled; write=78.637 s, sync=0.438 s, total=79.174 s; sync files=311, longest=0.010 s, average=0.002 s; distance=4304 kB, estimate=4304 kB; lsn=0/1DA21E8, redo lsn=0/1D7C1F8
2025-08-25 03:11:55.187 UTC [111] FATAL:  role "postgres" does not exist
2025-08-25 03:12:48.444 UTC [120] FATAL:  role "motoredocker" does not exist
2025-08-25 03:40:49.310 UTC [1] LOG:  received fast shutdown request
2025-08-25 03:40:49.314 UTC [1] LOG:  aborting any active transactions
2025-08-25 03:40:49.322 UTC [95] FATAL:  terminating connection due to administrator command
2025-08-25 03:40:49.322 UTC [93] FATAL:  terminating connection due to administrator command
2025-08-25 03:40:49.322 UTC [90] FATAL:  terminating connection due to administrator command
2025-08-25 03:40:49.322 UTC [89] FATAL:  terminating connection due to administrator command
2025-08-25 03:40:49.323 UTC [92] FATAL:  terminating connection due to administrator command
2025-08-25 03:40:49.334 UTC [1] LOG:  background worker "logical replication launcher" (PID 68) exited with exit code 1
2025-08-25 03:40:49.342 UTC [63] LOG:  shutting down
2025-08-25 03:40:49.346 UTC [63] LOG:  checkpoint starting: shutdown immediate
2025-08-25 03:40:49.421 UTC [63] LOG:  checkpoint complete: wrote 48 buffers (0.3%); 0 WAL file(s) added, 0 removed, 0 recycled; write=0.014 s, sync=0.041 s, total=0.078 s; sync files=37, longest=0.010 s, average=0.002 s; distance=152 kB, estimate=3889 kB; lsn=0/1DA2298, redo lsn=0/1DA2298
2025-08-25 03:40:49.431 UTC [1] LOG:  database system is shut down

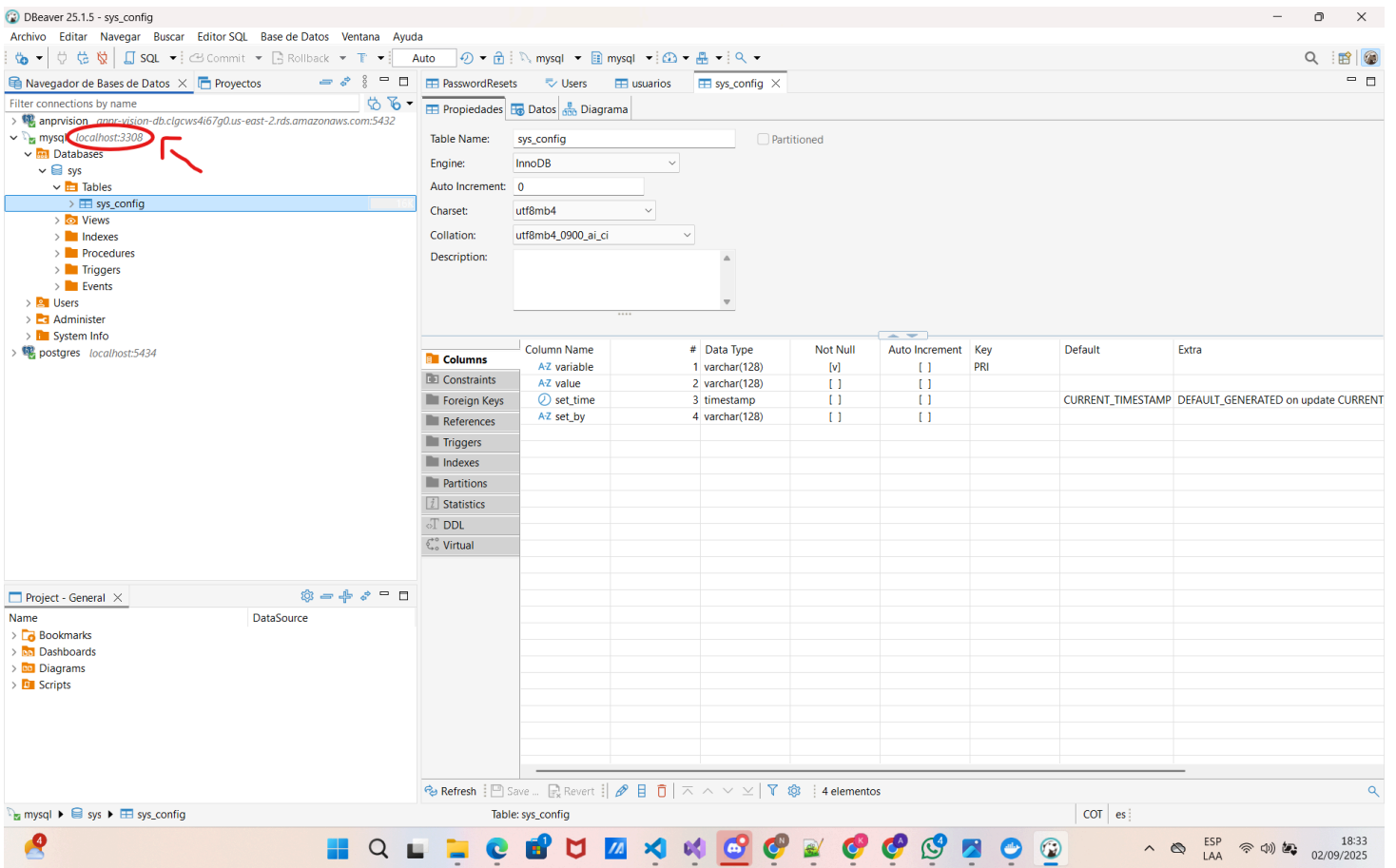
PostgreSQL Database directory appears to contain a database; Skipping initialization

2025-09-02 22:57:46.973 UTC [1] LOG:  starting PostgreSQL 17.6 (Debian 17.6-1.pgdg13+1) on x86_64-pc-linux-gnu, compiled by gcc (Debian 14.2.0-19) 14.2.0, 64-bit
2025-09-02 22:57:46.975 UTC [1] LOG:  listening on IPv4 address "0.0.0.0", port 5432
2025-09-02 22:57:46.975 UTC [1] LOG:  listening on IPv6 address ":::", port 5432
2025-09-02 22:57:46.986 UTC [1] LOG:  listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
2025-09-02 22:57:47.076 UTC [30] LOG:  database system was shut down at 2025-08-25 03:40:49 UTC
2025-09-02 22:57:47.112 UTC [1] LOG:  database system is ready to accept connections
2025-09-02 23:02:47.084 UTC [28] LOG:  checkpoint starting: time
2025-09-02 23:02:47.156 UTC [28] LOG:  checkpoint complete: wrote 3 buffers (0.0%); 0 WAL file(s) added, 0 removed, 0 recycled; write=0.029 s, sync=0.004 s, total=0.073 s; sync files=2, longest=0.002 s, average=0.002 s; distance=0 kB, estimate=0 kB; lsn=0/1DA23A0, redo lsn=0/1DA2348
```

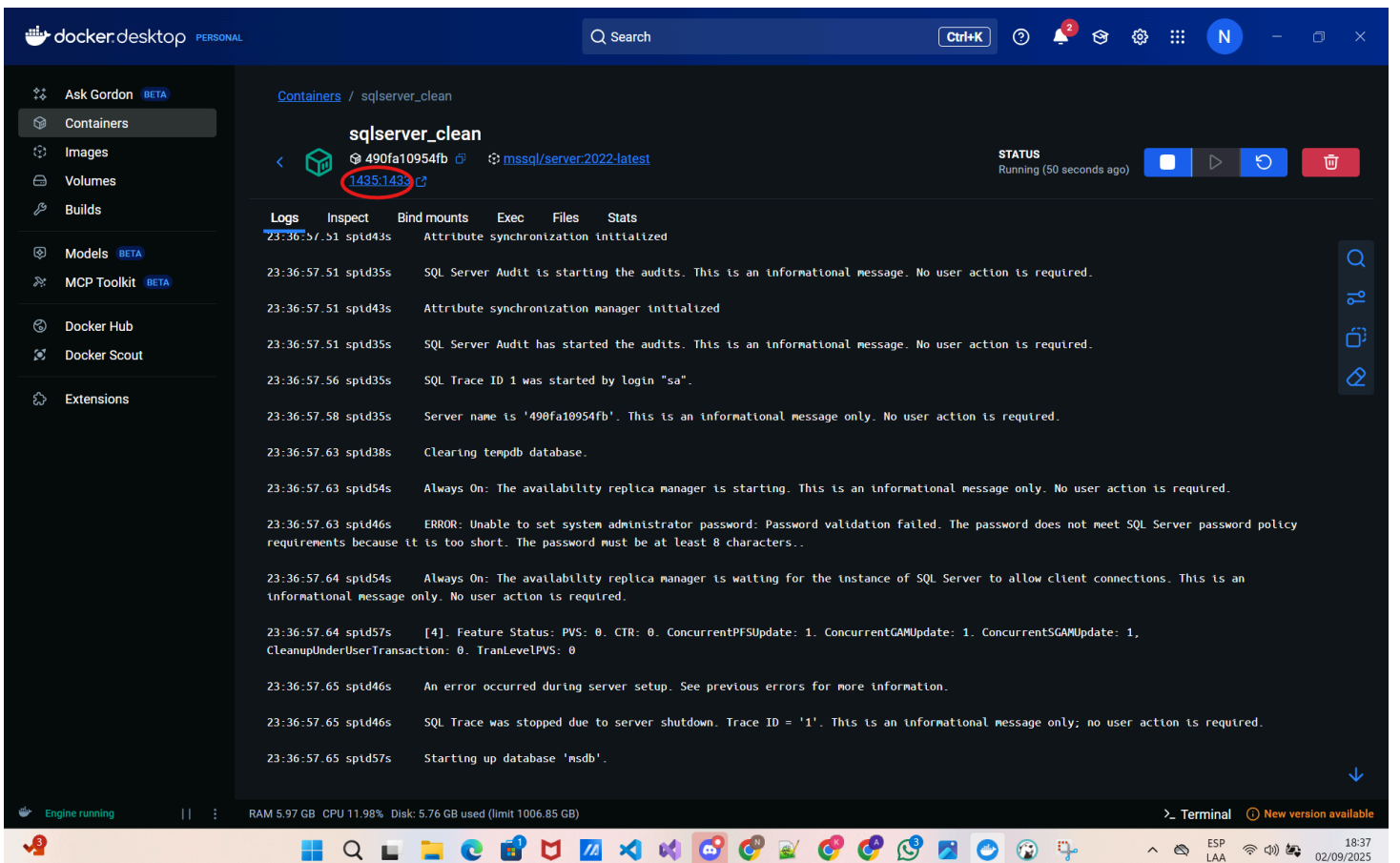


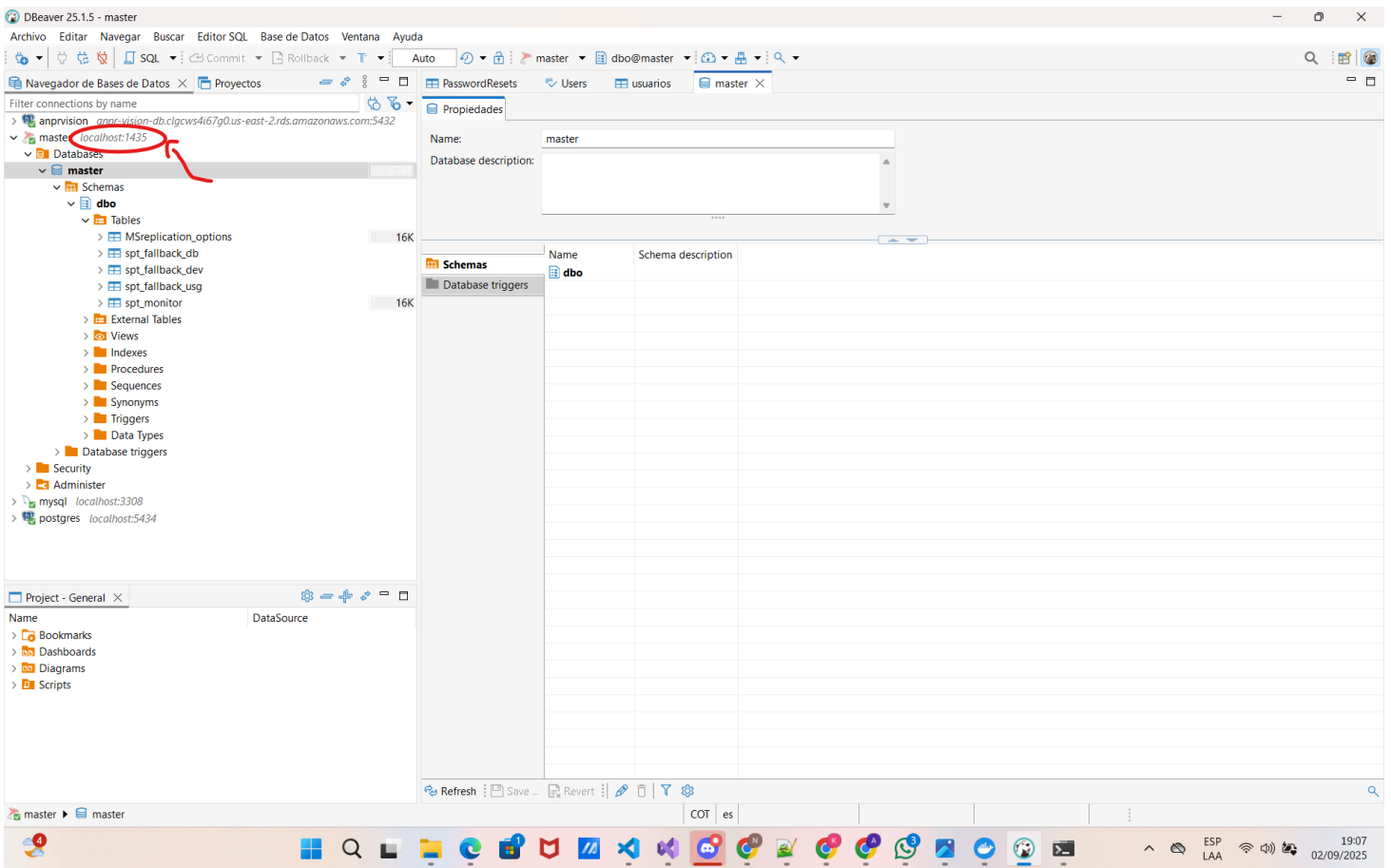
Conexión a MySQL





Conexión a SQL Server





2. Backend y Frontend en Docker apuntando a la base de datos Docker

Vista general: contenedores de apps y motores de BD: La captura de Docker Desktop muestra ejecutándose, al mismo tiempo, los contenedores del backend, del frontend y de los motores de base de datos. Esta vista confirma que todos los servicios están arriba, comparten la misma red de Docker y pueden comunicarse por nombre de servicio.

docker.desktop PERSONAL

Search Ctrl+K

Containers [Give feedback](#)

View all your running containers and applications. [Learn more](#)

Container CPU usage **4.18% / 1200%** (12 CPUs available)

Container memory usage **1.57GB / 7.4GB**

Show charts

Search

Only show running containers

	Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
☐	determined_noyce	dd287662e82a	postgres:latest		0%	8 hours ago	
☐	bold_kare	95dd29406e22	web-backend:latest		0%	4 hours ago	
☐	vibrant_davinci	80df49b7d38e	web-backend:latest		0%	4 hours ago	
☒	peaceful_wilbur	f5b3fa5981bd	web-backend:latest		0.09%	4 hours ago	
☒	motoresdocker	-	-	-	1.06%	7 hours ago	
☐	postgres_clean	cc288eccc00c	postgres:latest	5434:5432	0.01%	7 hours ago	
☐	mysql_clean	ac62a5e1c791	mysql:latest	3308:3306	1.05%	7 hours ago	
☒	api	-	-	-	3.03%	19 minutes ago	
☐	anpr-sql	8e9d4abdf697	mssql/server:2022-lates	1436:1433	2.93%	1 hour ago	
☐	anpr-api	a06b1a629334	api-api	5002:8080	0.1%	19 minutes ago	
☐	anpr-frontend	22222945efa6	api-frontend	4200:80	0%	19 minutes ago	

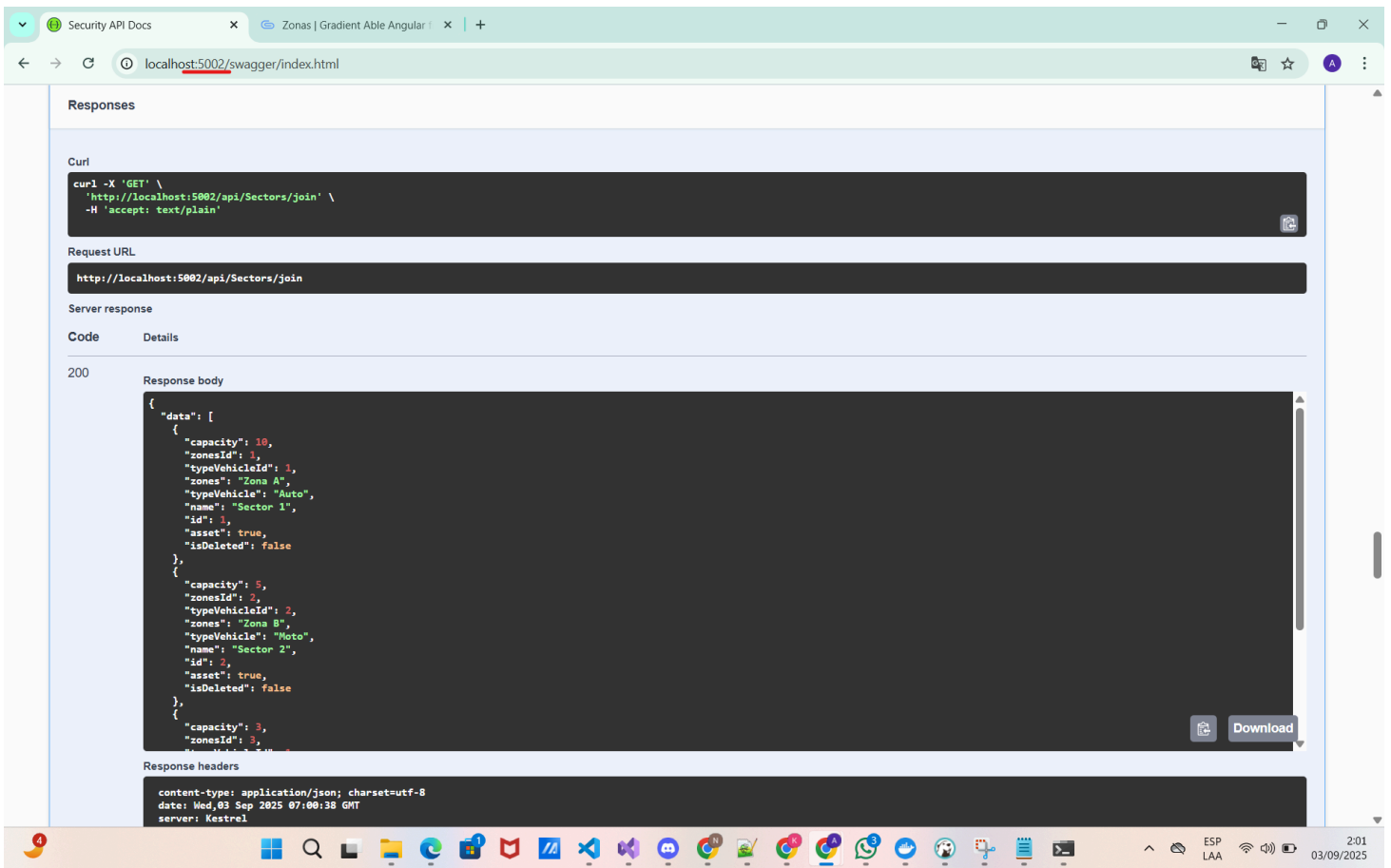
Showing 11 items

Engine running RAM 5.22 GB CPU 0.50% Disk: 18.93 GB used (limit 1006.85 GB)

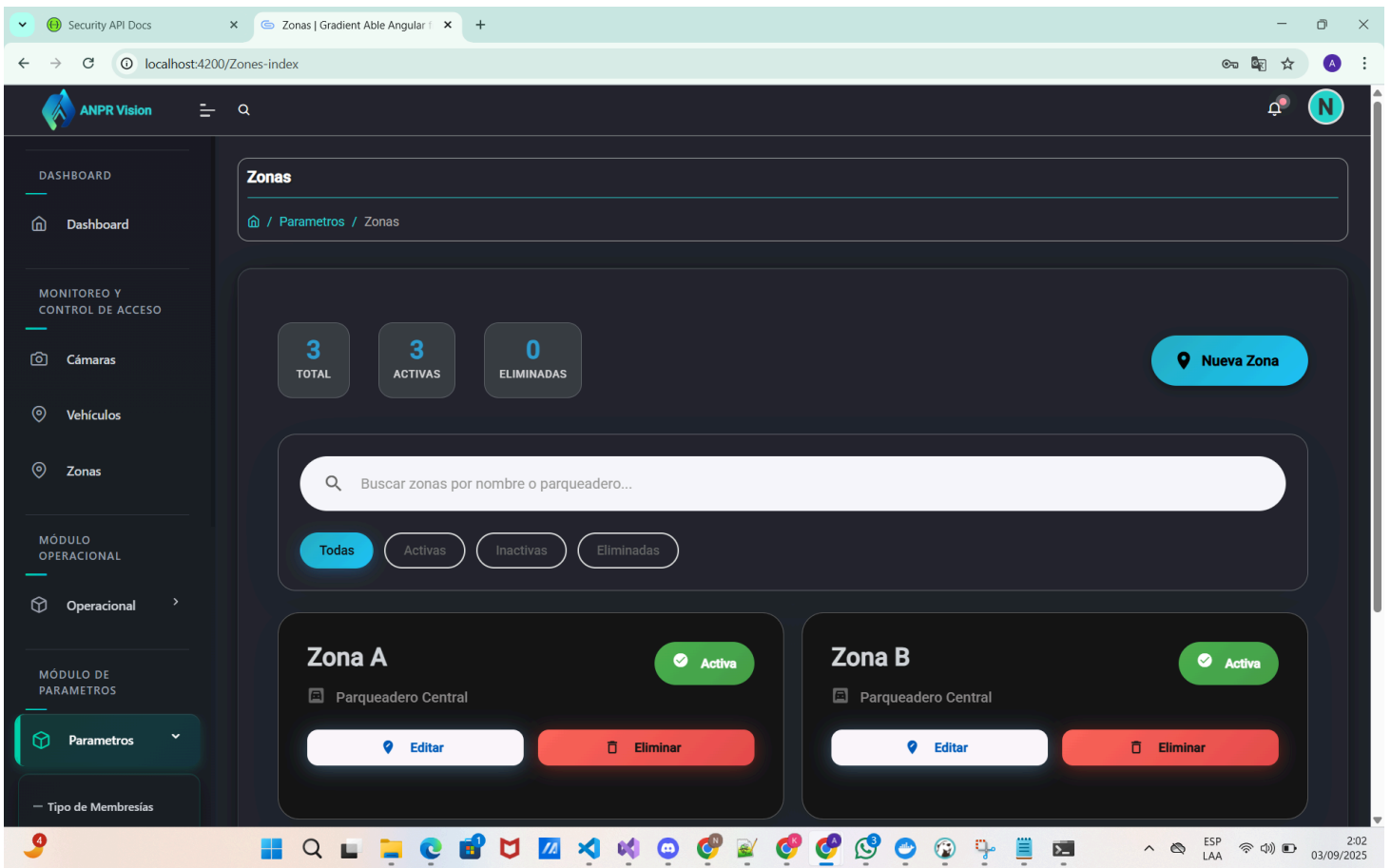
Terminal New version available

1:59 03/09/2025

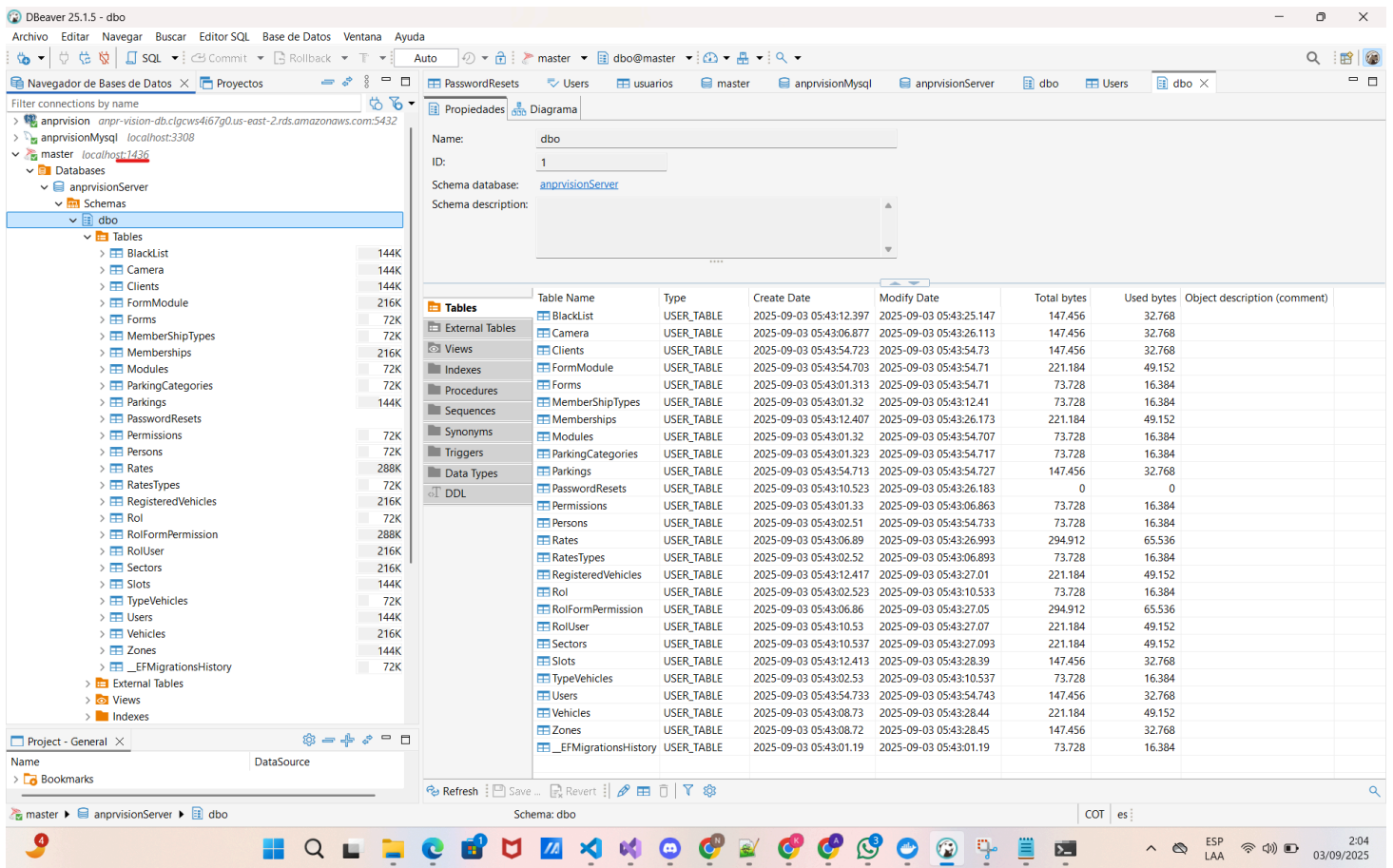
Prueba del backend con Swagger: En la sección inferior se abre Swagger UI de la API corriendo dentro del contenedor. Se ejecuta un endpoint (por ejemplo, `/api/Sectors/join`) y se observa el cURL, el Request URL y una respuesta 200 OK. Esto valida que el backend está correctamente expuesto hacia el host y que consulta la base de datos que también se ejecuta en Docker.



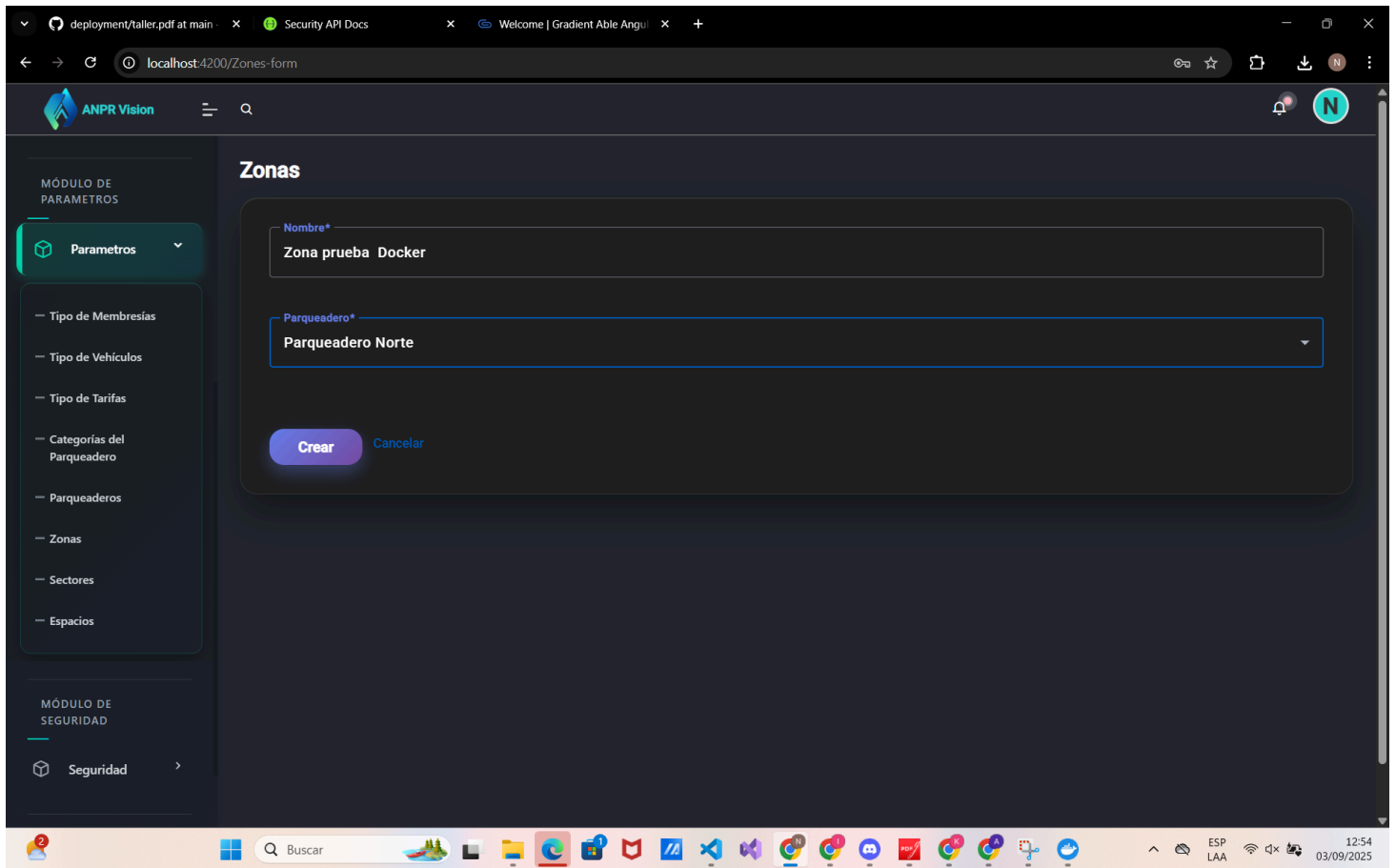
Frontend operativo y conectado al backend: La siguiente captura muestra el frontend en la pantalla de Zonas. Se visualizan tarjetas como Zona A y Zona B con sus estados, junto con filtros y acciones (Editar, Eliminar). Este resultado evidencia que el frontend está consumiendo la API y renderizando datos reales provenientes de la base de datos del entorno Docker.



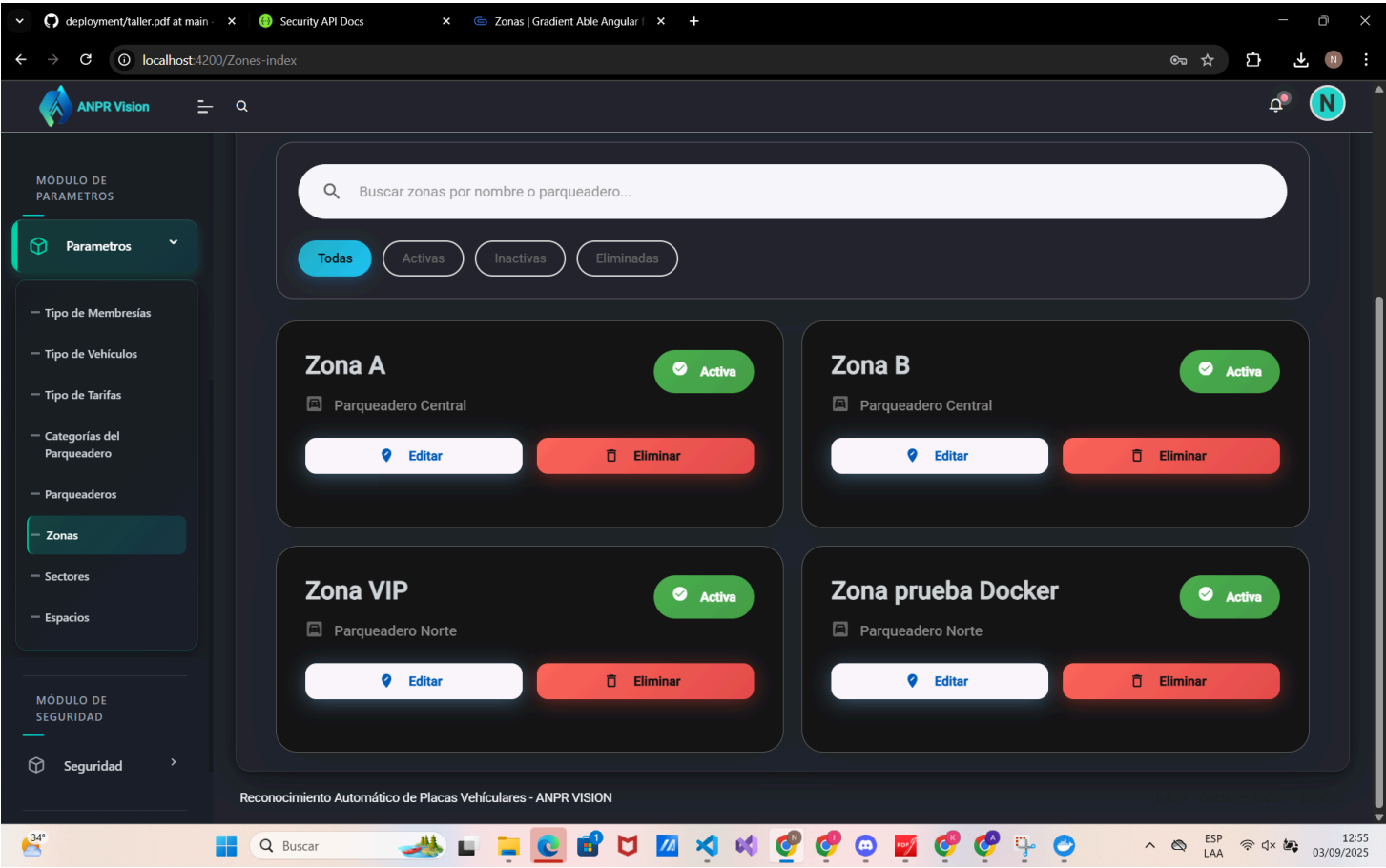
Verificación de datos en el cliente SQL: En la parte inferior se ve el cliente de base de datos (DBeaver) conectado al motor en Docker. Se inspeccionan esquemas y tablas del proyecto (por ejemplo, Zonas, Sectors, MembershipTypes, Rates, etc.). Esta comprobación cierra el circuito end-to-end: los datos mostrados en el frontend son los mismos que residen en la BD de Docker y que la API consulta en tiempo real.



Realizamos un registro nuevo desde el fronted apuntando al backend montado en docker.



Registro creado



Verificamos que el registro se ha creado

deployment/taller.pdf at mainSecurity API DocsZonas | Gradient Able Angular

localhost:5002/swagger/index.html

zone4/13

Responses

Curl

curl -X 'GET' \n'http://localhost:5002/api/Zones/join' \n-H 'accept: text/plain'

Request URL

http://localhost:5002/api/Zones/join

Server response

CodeDetails

200

Response body

```
{
  "message": "Ok",
  "parking": "Parqueadero Central",
  "name": "Zona B",
  "id": 2,
  "asset": true,
  "isDeleted": false
},
{
  "parkingId": 2,
  "parking": "Parqueadero Norte",
  "name": "Zona VIP",
  "id": 3,
  "asset": true,
  "isDeleted": false
},
{
  "parkingId": 2,
  "parking": "Parqueadero Norte",
  "name": "Zona prueba Docker",
  "id": 1001,
  "asset": true,
  "isDeleted": false
}
],
"success": true,
"message": "Ok",
"details": null
}
```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Wed, 03 Sep 2025 17:55:43 GMT
server: Kestrel
transfer-encoding: chunked
```

Windows Taskbar

Buscar

Icons: File Explorer, Edge, Mail, Teams, etc.

System Tray: ESP LAA, 12:55, 03/09/2025